

Line-Segment Properties Q&A

Section 33.1 Review Questions

Question 1

Prove that if $p_1 \times p_2$ is positive, then vector p_1 is clockwise from vector p_2 with respect to the origin $(0, 0)$, and that if this cross product is negative, then p_1 is counterclockwise from p_2 .

Answer 1

The sign determines relative rotation because the cross product can be written using the sine of the angle difference.

Write the two vectors in polar form:

$$p_1 = r_1(\cos \theta_1, \sin \theta_1), \quad p_2 = r_2(\cos \theta_2, \sin \theta_2).$$

Then

$$p_1 \times p_2 = r_1 r_2 (\cos \theta_1 \sin \theta_2 - \sin \theta_1 \cos \theta_2).$$

Using

$$\sin(\theta_2 - \theta_1) = \sin \theta_2 \cos \theta_1 - \cos \theta_2 \sin \theta_1,$$

we get

$$p_1 \times p_2 = r_1 r_2 \sin(\theta_2 - \theta_1).$$

Since $r_1 r_2 > 0$, the sign depends only on $\sin(\theta_2 - \theta_1)$.

If $p_1 \times p_2 > 0$, then $\theta_2 - \theta_1$ is positive for the smaller rotation, so p_2 is counterclockwise from p_1 . Equivalently, p_1 is clockwise from p_2 .

If $p_1 \times p_2 < 0$, then p_2 is clockwise from p_1 . Equivalently, p_1 is counterclockwise from p_2 .

So:

$$p_1 \times p_2 > 0 \Rightarrow p_1 \text{ is clockwise from } p_2,$$

$$p_1 \times p_2 < 0 \Rightarrow p_1 \text{ is counterclockwise from } p_2.$$

Question 2

Why is checking only the x -coordinate in ON-SEGMENT incorrect?

Answer 2

Checking only the x -coordinate is incorrect because a point can have the correct x -range but still lie outside the segment in the y -direction.

The correct on-segment test, assuming p_i, p_j, p_k are collinear, is

$$\min(x_i, x_j) \leq x_k \leq \max(x_i, x_j)$$

and

$$\min(y_i, y_j) \leq y_k \leq \max(y_i, y_j).$$

Counterexample:

$$p_i = (0, 0), \quad p_j = (0, 2), \quad p_k = (0, 5).$$

The x -test passes:

$$0 \leq 0 \leq 0.$$

But the y -test fails:

$$0 \leq 5 \leq 2$$

is false. Therefore p_k is on the same vertical line but outside the segment.

Both x and y ranges must be checked.

Question 3

The polar angle of a point p_1 with respect to an origin point p_0 is the angle of vector $p_1 - p_0$ in the usual polar coordinate system. For example, the polar angle of $(3, 5)$ with respect to $(2, 4)$ is the angle of the vector $(1, 1)$, which is 45° or $\pi/4$ radians. The polar angle of $(3, 3)$ with respect to $(2, 4)$ is the angle of vector $(1, -1)$, which is 315° or $7\pi/4$ radians. Write pseudocode to sort a sequence $\langle p_1, p_2, \dots, p_n \rangle$ of n points according to their polar angles with respect to a given origin point p_0 . Your procedure should take $O(n \lg n)$ time and use cross products to compare angles.

Answer 3

The solution is to translate all points so that p_0 becomes the origin, then sort the translated vectors with a comparator based on half-plane and cross product.

Step 1: Translate the Points

For every point p_i , compute

$$v_i = p_i - p_0.$$

Now sorting points by polar angle around p_0 is the same as sorting vectors v_i by polar angle around $(0, 0)$.

For the examples in the question:

$$(3, 5) - (2, 4) = (1, 1),$$

$$(3, 3) - (2, 4) = (1, -1).$$

Thus $(3, 5)$ has angle 45° , while $(3, 3)$ has angle 315° .

Step 2: Split the Circle into Two Parts

A cross product alone cannot correctly order vectors across the 0° boundary. So first classify each vector:

$$\text{Half}(v) = \begin{cases} 0, & v_y > 0 \text{ or } (v_y = 0 \text{ and } v_x \geq 0), \\ 1, & v_y < 0 \text{ or } (v_y = 0 \text{ and } v_x < 0). \end{cases}$$

Vectors with $\text{Half}(v) = 0$ come before vectors with $\text{Half}(v) = 1$. This gives the usual polar-angle order:

$$0^\circ, 45^\circ, 90^\circ, \dots, 180^\circ, \dots, 315^\circ.$$

Step 3: Compare Vectors in the Same Half

For two vectors

$$u = (u_x, u_y), \quad v = (v_x, v_y),$$

compute

$$u \times v = u_x v_y - u_y v_x.$$

If u and v are in the same half:

$$u \times v > 0 \Rightarrow u \text{ comes before } v,$$

$$u \times v < 0 \Rightarrow v \text{ comes before } u.$$

If $u \times v = 0$, the points have the same polar angle. Break the tie using squared distance:

$$\|u\|^2 = u_x^2 + u_y^2.$$

Comparator Example

Let

$$u = (1, 1), \quad v = (1, -1).$$

Then

$$\text{Half}(u) = 0, \quad \text{Half}(v) = 1.$$

So u comes before v , matching

$$45^\circ < 315^\circ.$$

Polar Angle Sort

```
PolarAngleSort(points, p0):
    for each point pi:
        vi = pi - p0

    sort vectors using Compare(u, v)

Compare(u, v):
    if Half(u) < Half(v):
        return u comes first

    if Half(u) > Half(v):
        return v comes first

    cross = u.x * v.y - u.y * v.x

    if cross > 0:
        return u comes first

    if cross < 0:
        return v comes first

    return shorter squared distance comes first
```

The translation step takes $O(n)$. Sorting n vectors takes $O(n \lg n)$, and each comparison takes $O(1)$. Therefore the total running time is

$$O(n \lg n).$$

Question 4

How can we determine whether any three points in a set of n points are collinear in $O(n^2 \lg n)$ time?

Answer 4

Use each point once as a temporary origin, sort the other points by polar angle around it, and look for equal adjacent angles.

- Choose a point p_i as the origin.
- For every other point p_j , compute the vector

$$v_j = p_j - p_i.$$

- Sort all vectors v_j by polar angle.
- After sorting, vectors with the same polar angle become adjacent.
- For every adjacent pair v_j, v_k , check

$$v_j \times v_k = 0.$$

- If this happens, then

p_i, p_j, p_k are collinear.

- Repeat this process for every possible origin p_i .

Detect Three Collinear Points

```
AnyThreeCollinear(points):  
    for each point pi in points:  
        V = empty list  
  
        for each point pj != pi:  
            add vector (pj - pi) to V  
  
        sort V by polar angle  
  
        for k = 1 to length(V) - 1:  
            if V[k-1] cross V[k] == 0:  
                return true  
  
    return false
```

- Building vectors for one origin takes $O(n)$.
- Sorting them takes $O(n \lg n)$.
- Scanning adjacent pairs takes $O(n)$.
- So one origin costs $O(n \lg n)$.
- There are n choices of origin.

Therefore:

$$n \cdot O(n \lg n) = O(n^2 \lg n).$$

Question 5

How can we test whether a polygon is convex in linear time, and why is checking only for mixed left/right turns not enough?

Answer 5

A polygon is convex only if all consecutive turns have the same nonzero sign. Checking only that there is no mixture of left and right turns can fail when a zero turn appears.

A zero turn means three consecutive vertices are collinear:

$$\text{orient}(p_i, p_{i+1}, p_{i+2}) = 0.$$

Then the middle vertex lies on the segment between its neighbors and is not a true corner. Under the strict vertex condition used in the textbook, that should be rejected.

Convex Polygon Test

```
IsConvex(P):
    expected = NONE

    for every consecutive triple (pi, p(i+1), p(i+2)):
        value = orientation(pi, p(i+1), p(i+2))

        if value == 0:
            return false

        if expected == NONE:
            expected = sign(value)
        else if sign(value) != expected:
            return false

    return true
```

Only one pass over the vertices is needed:

$$\Theta(n).$$

Convex polygon = same nonzero turn direction at every vertex.

Question 6

How can a right horizontal ray from p_0 be tested for intersection with a segment in $O(1)$ time?

Answer 6

Replace the infinite ray by a finite horizontal segment that is long enough to cover every possible intersection with $\overline{p_1 p_2}$. Then use the ordinary segment-intersection algorithm.

Step 1: Understand the Ray

The right horizontal ray starting from

$$p_0 = (x_0, y_0)$$

contains all points

$$(x, y_0) \quad \text{where } x \geq x_0.$$

The segment we want to test is

$$\overline{p_1 p_2}, \quad p_1 = (x_1, y_1), \quad p_2 = (x_2, y_2).$$

Step 2: Choose a Far-Right Endpoint

choose

$$p_3 = (\max(x_0, x_1, x_2) + 1, y_0).$$

This point lies on the same horizontal line as p_0 , but strictly to the right of p_0, p_1 , and p_2 in the x -direction. Therefore, if the right ray hits $\overline{p_1 p_2}$, that hit must occur before or at p_3 .

So the ray can be replaced by the finite segment

$$\overline{p_0 p_3}.$$

Step 3: Reduce to Segment Intersection

Now test whether

$$\overline{p_0 p_3}$$

intersects

$$\overline{p_1 p_2}.$$

Thus:

$$\text{ray from } p_0 \text{ intersects } \overline{p_1 p_2} \iff \overline{p_0 p_3} \text{ intersects } \overline{p_1 p_2}.$$

Small Example

Suppose

$$p_0 = (2, 3), \quad p_1 = (4, 1), \quad p_2 = (6, 5).$$

Then

$$\max(x_0, x_1, x_2) = \max(2, 4, 6) = 6.$$

So choose

$$p_3 = (6 + 1, 3) = (7, 3).$$

Now test whether

$$\overline{(2, 3)(7, 3)}$$

intersects

$$\overline{(4, 1)(6, 5)}.$$

Since segment intersection uses a constant number of orientation and on-segment tests, the running time is

$$O(1).$$

Question 7

How can we determine whether a point lies inside a simple polygon in $\Theta(n)$ time?

Answer 7

Use the ray-casting, or odd-even, rule. Cast a horizontal ray from the query point and count how many times it crosses the polygon boundary.

First check whether the query point p_0 lies on any polygon edge. If it does, return boundary. Otherwise, scan every edge exactly once. For edge

$$\overline{p_i p_{i+1}},$$

use the half-open vertical straddling rule:

$$(y_i > y_0) \neq (y_{i+1} > y_0).$$

This identifies edges that cross the horizontal level of p_0 without double-counting vertices.

If the crossing point is to the right of p_0 , toggle the crossing parity.

At the end:

$$\boxed{\text{odd number of crossings} \Rightarrow \text{inside},}$$

$$\boxed{\text{even number of crossings} \Rightarrow \text{outside}.}$$

Each of the n edges is processed once, so the time is

$$\boxed{\Theta(n).}$$

Question 8

How can the area of an n -vertex simple polygon be computed in $\Theta(n)$ time?

Answer 8

Use the shoelace formula, which sums cross products of consecutive vertices.

Let the vertices in boundary order be

$$p_i = (x_i, y_i), \quad i = 0, 1, \dots, n-1,$$

with wraparound

$$p_n = p_0.$$

The signed area is

$$A_s = \frac{1}{2} \sum_{i=0}^{n-1} p_i \times p_{i+1}.$$

Since

$$p_i \times p_{i+1} = x_i y_{i+1} - x_{i+1} y_i,$$

the actual polygon area is

$$\boxed{A = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|}.$$

Polygon Area

```
PolygonArea(P):  
    sum = 0  
  
    for i = 0 to n - 1:  
        j = (i + 1) mod n  
        sum = sum + xi*yj - xj*yi  
  
    return abs(sum) / 2
```

The loop scans the vertices once:

$$\Theta(n).$$